

Mini Project: Build and Audit an MCP Server

Extend a server with your own tool and knowledge, measure what it tells an agent, and say what a protocol gives you that a plain function does not.

Runs on your own machine or Colab • 100 points

1. What this project is about

MCP is a contract. A server advertises tools, and a client, or a model acting through one, discovers them and calls them, trusting whatever comes back. So the interesting question is not whether you can build a server. It is what happens when a tool is quietly wrong, and how much an agent should trust an answer it cannot check.

Part 1 is practice: work through the two lecture notebooks in this module, `mcp_build_server` and `mcp_client_agent`, then finish five short pieces of code, some in the server and some in the client. The rest is the project proper. You extend the server with a host, a tool, and knowledge of your own, measure what it tells a client, and report what you found.

Everything runs offline: simulated scans, a local knowledge base, no model, no API key, and no traffic on the network, so your numbers are exact and reproducible.

2. Using AI assistants

You may use ChatGPT, Claude, Copilot, or any other assistant to write code, fix errors, and organize results. What is assessed is whether you understand what you built: in the video you will walk through your own server and client and explain what a client sees when it connects. Anything you cannot explain will not earn credit.

3. Part 1: practice on both sides of MCP

10 points

MCP has two sides, so the practice pieces live in two files. Work from the two lecture notebooks: `mcp_build_server` covers tools and resources, and `mcp_client_agent` covers connecting, discovering, and calling.

	File	You write	Lines
TODO 1	<code>project_mcp_server.py</code>	a version aware CVE match	2
TODO 2	<code>project_mcp_server.py</code>	expose the knowledge base as a resource	1
TODO 3	<code>MiniProject_MCP_Starter.ipynb</code>	connect over stdio and discover the tools	3
TODO 4	<code>MiniProject_MCP_Starter.ipynb</code>	call a tool	3
TODO 5	<code>MiniProject_MCP_Starter.ipynb</code>	read the resource	3

The original lookup tool from the lecture stays in the server untouched, so you can measure your version against it.

4. Part 2: your own server and your own tool

30 points

From Part 8 of the notebook onward, no code is provided. Your work here spans two files, the server and the notebook, and both are submitted.

Extend the server in `project_mcp_server.py`. Add one host of your own to `_SIMULATED_HOSTS` with three realistic service banners, chosen so that at least one is covered by the knowledge base and at least one is not. Add a tool of your own that an analyst would find useful, such as one that reports the severity of a finding, checks a port against a policy list, or summarizes what a host exposes. Write its docstring as carefully as its code, because the docstring is what a client and a model read when they decide whether to call it.

Add the knowledge that is missing. Some services return nothing because the knowledge base has no entry, and no matching rule fixes that. Write real entries for real products with real CVEs, cite them in `SOURCES.md`, and record the expected section for each service.

Then write the client side yourself, in the notebook: connect to your server, discover the tools, and show that your new tool is advertised with its description. The client launches the server as a subprocess, so an edit to the server file is live the next time you run a client cell.

5. Part 3: measure what the server tells a client

15 points

Sweep your own host with both lookup tools and with your new tool, and sort every answer into four buckets.

	Meaning	Why it matters
grounded	the right advisory came back	what you want
blind spot	no known CVEs, but the knowledge base does cover it	the agent is reassured, wrongly
wrong version	an advisory for a version this host is not running	worse than silence, because it looks authoritative
true gap	no known CVEs, and the knowledge base really has nothing	honest, and fixed with knowledge rather than code

Save the table as `results_mine.csv`. Then make one knowledge base entry deliberately vague by dropping the version, run the sweep again, and record what changed. Better matching and better knowledge are two different levers, and this is how you see them trade against each other.

6. What to hand in

Submit this

one PDF report. On its first page, put the link to your project folder on GitHub and the link to your video. Everything else lives in the repository.

6.1 Your repository (5 points)

Use the private repository from the earlier projects, with your instructor and the class TA as collaborators, and add a new folder for this one:

```
your-repo/
  mcp_project/
    README.md          what the project is, in a short paragraph
    mcp_notebook.ipynb your notebook, with all outputs saved
    project_mcp_server.py your extended server
    project_cve_kb.md   the knowledge base, with your entries
    expected_cves.json  the expected section for each service
    SOURCES.md          your citations and your results
    results_mine.csv
```

Run the notebook top to bottom and save it with its outputs showing before you commit.

6.2 Report (25 points)

A PDF. Begin with the concept check, five short answers of two or three sentences each:

- What is the many to many integration problem, and how does MCP turn it into many plus many?
- What are the host, the client, and the server in MCP? Which one exposes capabilities?
- What is the difference between a tool, a resource, and a prompt? Give a security example of each.
- What does the stdio transport do, and how does it differ from HTTP?
- How does a client discover what a server offers, and how does it call a tool?

Then the analysis:

1. What you built, briefly: your matching rule, your new tool, the knowledge you added, and the host you invented.
2. A blind spot compared with a wrong answer. Why is silence about a covered service more dangerous than an error, which failure would you rather ship, and what does that imply about trusting a tool result?
3. The two levers. What did better matching fix, what needed new knowledge, and what happened when you made an entry vague on purpose? Give the numbers before and after.
4. The docstring is the contract. Which description on your server would you rewrite before letting a model use it, and which tool would you refuse to expose at all?
5. A tool over MCP compared with a tool inside an agent. In the LangGraph module a tool was a Python function you called yourself. Here it is a separate process that a client discovers at run time. What did you gain, what did it cost, and when would you keep the plain function?
6. Honest limits. A handful of services on invented hosts, and a knowledge base you partly wrote yourself. What can this not tell you?

6.3 Video (15 points)

Five to ten minutes of screen recording with your narration. Start the server, show a client connecting and discovering the tools, call your own tool, and talk through your measurement. Live screens, not slides.

7. Grading

Part	Points
Part 1, practice: the five pieces complete, on both the server and the client	10
Part 2, your host, your tool with its docstring, and your knowledge base entries	30
Part 3, the four buckets measured, the vague entry experiment, results_mine.csv	15
Repository: private, instructor and TA added, separate folder, outputs saved	5
Report, including the concept check and the comparison with a tool inside an agent	25
Video	15
Total	100

Keep every tool simulated or read only, never scan a host you are not authorized to test, and cite real CVEs rather than inventing advisories.